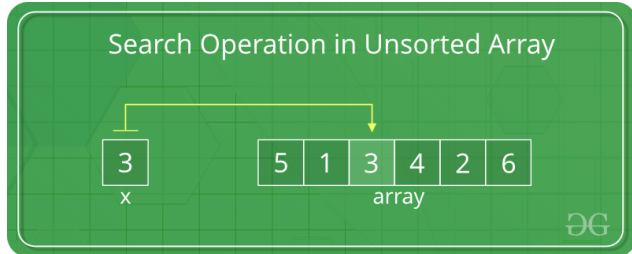


Search, Insert, and Delete in an Unsorted Array

Search Operation:

In an unsorted array, the search operation can be performed by linear traversal from the first element to the last element.



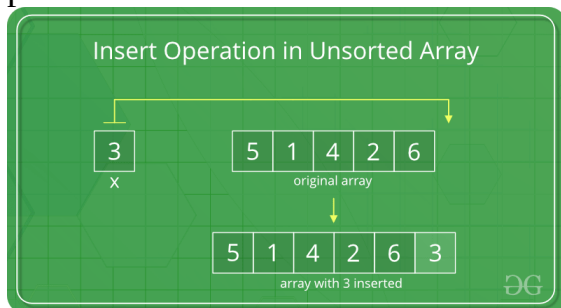
```
int findElement(int arr[], int n, int key)
{
    int i;
    for (i = 0; i < n; i++)
        if (arr[i] == key)
            return i;

    // If the key is not found
    return -1;
}
```

Insert Operation:

1. Insert at the end:

In an unsorted array, the insert operation is faster as compared to a sorted array because we don't have to care about the position at which the element is to be placed.



```
int insertSorted(int arr[], int n, int key, int capacity)
{
    // Cannot insert more elements if n is
    // already more than or equal to capacity
    if (n >= capacity)
```

```

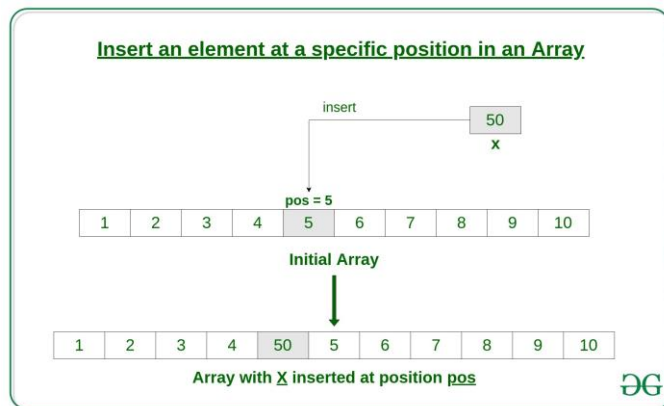
    return n;

    arr[n] = key;
    return (n + 1);
}

```

2. Insert at any position

Insert operation in an array at any position can be performed by shifting elements to the right, which are on the right side of the required position



```

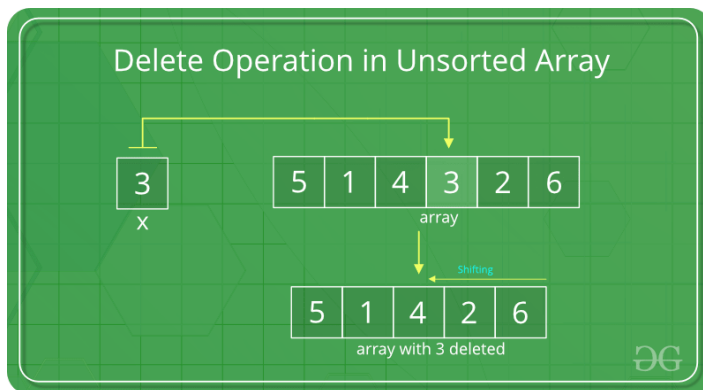
void insertElement(int arr[], int n, int x, int pos)
{
    // shift elements to the right
    // which are on the right side of pos
    for (int i = n - 1; i >= pos; i--)
        arr[i + 1] = arr[i];

    arr[pos] = x;
}

```

Delete Operation:

In the delete operation, the element to be deleted is searched using the linear search, and then the delete operation is performed followed by shifting the elements.



```
int deleteElement(int arr[], int n, int key)
{
    // Find position of element to be deleted
    int pos = findElement(arr, n, key);

    if (pos == -1) {
        cout << "Element not found";
        return n;
    }

    // Deleting element
    int i;
    for (i = pos; i < n - 1; i++)
        arr[i] = arr[i + 1];

    return n - 1;
}

int findElement(int arr[], int n, int key)
{
    int i;
    for (i = 0; i < n; i++)
        if (arr[i] == key)
            return i;

    return -1;
}
```

Rearrange and update array elements as specified by the given queries

Given an array **arr[]** of size **N** and queries **Q[][]**, the task is to perform the following types of queries on the given array. **0**: Left shift the array by one position.

- **1**: Right shift the array by one position.
- **2 X Y**: Update the value of **arr[X] = Y**.
- **3 X**: Print **arr[X]**.

- **Example:**

- **Input:** **arr[]**={1, 2, 3, 4, 5}, **Q[][]**={{0}, {1}, {3, 1}, {2, 2, 54}, {3, 2}}.

Output: 4 54

Explanation:

Query1: The array **arr[]** modifies to {2, 3, 4, 5, 1}

Query2: The array **arr[]** modifies to {1, 2, 3, 4, 5}

Query3: Print the value of **arr[1]** i.e. 2

Query4: The array **arr[]** modifies to {1, 54, 3, 4, 5}

Query5: Print the value of **arr[2]**, i.e. 54.

- **Input:** **arr[]**={1}, **Q[][]**={{0}, {1}, {2, 0, 54}, {3, 0}}

Output: 54

Approach: The problem can be solved using Deque(Double Ended queue) to perform the insert and delete operation at the front and back of the queue in O(1). Follow the below steps to solve the problem.

1. Create a double-ended Queue, **dq**.
2. Push all elements of the array **arr[]** to **dq**.
3. For the query of type **0**(Left Shift), pop an element from the front of **dq** and push the element to the back of **dq**.
4. For the query of type **1**(Right Shift), pop an element from the back of **dq** and push the element to the front of **dq**.
5. For the query of type **2**, update **dq[X] = Y**.
6. For the query of type **3**, print **dq[X]**.

```
void Queries(int arr[], int N)
{
```

```
    // Dequeue to store the
    // array elements
    deque<int> dq;
```

```
    // Insert all element of
    // the array into the dequeue
```

```

for (int i = 0; i < N; i++) {
    dq.push_back(arr[i]);
}

// Stores the size of the queue
int sz = Q.size();

// Traverse each query
for (int i = 0; i < sz; i++) {

    // Query for left shift.
    if (Q[i][0] == 0) {

        // Extract the element at
        // the front of the queue
        int front = dq[0];

        // Pop the element at
        // the front of the queue
        dq.pop_front();

        // Push the element at
        // the back of the queue
        dq.push_back(front);
    }

    // Query for right shift
    else if (Q[i][0] == 1) {

        // Extract the element at
        // the back of the queue
        int back = dq[N - 1];

        // Pop the element at
        // the back of the queue
        dq.pop_back();

        // Push the element at
        // the front of the queue
        dq.push_front(back);
    }
}

```

```
}

// Query for update
else if (Q[i][0] == 2) {

    dq[Q[i][1]] = Q[i][2];
}

// Query to get the value
else {
    cout << dq[Q[i][1]] << " ";
}
}
```